

Programação Orientada a Objetos

Guia de Estudos com base nos slides do professor
e no código do projeto **Randongeon** (RPG em Python)

Os 4 pilares — Encapsulamento, Herança, Polimorfismo e Abstração — explicados, com dicas, e comparados com o código real do jogo. Inclui os recursos que o Python adiciona e o conteúdo que o projeto usou ALÉM dos slides.

Disciplina: Laboratório de Programação (POO) — UNIT Aracaju

Sumário

Parte	Conteúdo
1	O que é POO: classes, objetos, <code>__init__</code> /self, atributos de instância vs de classe
2	Os 4 pilares (definição do professor + código do Randongeon + dicas)
3	O que o Python adiciona: <code>@property</code> , duck typing, dunder, <code>@staticmethod</code> / <code>@classmethod</code> , herança múltipla/MRO, composição vs herança
4	Conteúdo ALÉM dos slides: o que o projeto usou que não foi ensinado
5	Crônica do projeto por lote (o que a pasta docs/ aborda)
6	Resumo dos pilares + dicas finais de estudo

Convenção: trechos de código vêm da pasta randongeon/jogo e api/ do projeto. Onde o código cita 'slide X', é referência direta ao material da aula.

Parte 1 — O que é POO

A **Programação Orientada a Objetos** é um modo de organizar o software em torno de **objetos** — entidades que combinam **dados** (atributos) e **comportamentos** (métodos). Contrasta com a programação procedural, focada em sequências de instruções. Benefícios: reutilização de código, organização, flexibilidade e facilidade de manutenção (alta coesão, baixo acoplamento).

Procedural vs Orientado a Objetos

No procedural, os dados andam soltos e as funções os recebem como parâmetro. Na POO, dados e comportamento ficam **juntos** dentro da classe:

```
# Procedural
nome = "Fusca"
velocidade = 0
def acelerar(v, inc):
    return v + inc
velocidade = acelerar(v, 20)

# Orientado a Objetos
class Carro:
    def __init__(self, nome):
        self.nome = nome
        self.velocidade = 0
    def acelerar(self, inc):
        self.velocidade += inc
```

Classe e Objeto

A **classe** é o molde (blueprint); o **objeto** é uma instância concreta criada a partir dela. Cada objeto tem os seus próprios valores de atributos. No Randomeon, Jogador, Inimigo e Item são classes; cada herói/monstro em jogo é um objeto.

O construtor `__init__` e o `self`

O `__init__` é chamado automaticamente ao criar o objeto e inicializa os atributos. O `self` é sempre o primeiro parâmetro e representa o próprio objeto (o Python o passa sozinho). Exemplo real do jogo:

```
class Jogador(Entidade):
    def __init__(self, nome, hp=20, atk=5, xp=0, esq=0.3, moedas=0):
        super().__init__(nome, hp) # a base valida nome/hp
        self.atk = atk # atributo de INSTANCIA
        self.xp = xp
        self.nivel = 1
        self.esq = esq
```

Atributos de instância vs de classe

Instância (`self.attr`): único de cada objeto — dados que variam (hp, atk, xp). **Classe** (`Cls.attr`): compartilhado por todos — ideal para **constantes**. O Randomeon usa atributos de classe para as constantes de balanceamento:

```
class Jogador(Entidade):
    ATK_POR_NIVEL = 2 # constantes de CLASSE (valem para todo jogador)
    HP_POR_NIVEL = 12
    XP_BASE_NIVEL = 10
    CURA_NIVEL_FRACAO = 0.60
    # ... self.hp, self.atk sao de INSTANCIA (variam por objeto)
```

Dica: Se o valor é igual para todos os objetos e nunca muda por objeto (uma regra/constante), ele pertence à CLASSE. Se varia de objeto para objeto (estado), pertence à INSTÂNCIA. No jogo, isso deixou os números de balanceamento todos juntos e fáceis de calibrar.

Parte 2 — Os 4 Pilares da POO

Para cada pilar: a **definição do professor** (slide), **como aparece no Randomeon** (com código real) e uma **dica** de estudo.

1) Abstração

Professor: ignorar detalhes não relevantes e focar nas características essenciais. Esconder o COMO e expor o O QUÊ. Em Python usa-se **classes abstratas (ABC)** com `@abstractmethod` para definir um contrato.

No Randomeon: a classe `Entidade(ABC)` é a base de tudo que combate. Ela define a interface comum (tem vida, cura, sofre dano) mas **não pode ser instanciada** — `receber_dano` é abstrato, cada filho implementa.

```
from abc import ABC, abstractmethod
class Entidade(ABC):
    # abstrata: nao instanciavel
    def esta_vivo(self): return self.hp > 0
    def curar(self, q): ... # comportamento COMUM
    @abstractmethod
    def receber_dano(self, dano): # CONTRATO: cada subclasse implementa
        raise NotImplementedError
```

Quem combate trata Jogador e Inimigo uniformemente, "como Entidades", sem saber o tipo concreto.

Dica: Use ABC quando faz sentido ter uma base mas NÃO criar objetos dela, e quando quer obrigar todas as subclasses a implementar certos métodos. Tentar instanciar uma ABC com método abstrato gera `TypeError` — o Python protege o contrato por você.

2) Encapsulamento

Professor: agrupar dados e métodos numa classe e **controlar o acesso** aos dados, protegendo o estado interno. Três níveis: público (acesso livre), `_protegido` (convenção, uso interno) e `__privado` (name mangling). Mais o `@property` (getter/setter com validação).

No Randomeon: usa-se **público** (`self.hp`), **_protegido** para detalhes internos (`_atualizar_nivel`, `_rolar_loot`, `_chance_item`) e **@property** para valores derivados de leitura:

```
class Jogador(Entidade):
    def _atualizar_nivel(self): # _protegido: detalhe interno, nao e API publica
        ...
    @property
    def pontuacao(self): # getter calculado (sem expor o calculo)
        return self.xp + (self.nivel - 1) * 50 + self.moedas
    @property
    def veneno_turnos(self): # derivado do efeito ativo (Lote B2)
        e = self.buscar_efeito("veneno")
        return e.turnos if e else 0
```

Dica: `@property` deixa um método ser lido como se fosse atributo (`jogador.pontuacao`, sem os parênteses), mantendo o cálculo escondido e o valor protegido contra escrita externa. É o jeito Pythonico de fazer getter sem poluir a classe com `get_pontuacao()`.

3) Herança

Professor: uma classe filha herda atributos e métodos da classe pai, evitando repetição. Em Python: `class Filha(Pai)`, e `super()` chama o pai. A filha pode **sobrescrever** métodos.

No Randomeon: a hierarquia é rica. Jogador e Inimigo herdam de `Entidade`; e o Inimigo tem 8 subclasses especializadas:

```
class Nosferatu(Inimigo):
    # "e um" Inimigo
    def __init__(self, bonus_hp=0, bonus_atk=0):
        super().__init__(nome="Nosferatu", cura_percentual=0.20, ...) # reusa o pai
    def tabela_loot(self):
        # sobrescreve so o que muda
        return LOOT_NOSFERATU
```

Subclasses: Nosferatu, GolemDePedra, Banshee, Orc, TrollDasCavernas, HordaDeGoblins, Goblin, CoracaoDaMasmorra. Todas reaproveitam o combate da base via `super()`.

Dica: Herança é a relação "É UM": Goblin É UM Inimigo. Se você se pegar copiando código entre duas classes parecidas, provavelmente existe uma classe-pai esperando para nascer. Mas cuidado: nem tudo é herança — veja Composição na Parte 3.

4) Polimorfismo

Professor: "muitas formas" — o mesmo método se comporta diferente conforme o tipo do objeto. Sobrescrita (overriding): a subclasse dá a sua própria versão de um método herdado.

No Randomeon: o caso mais elegante é o combate: um único laço serve a TODOS os inimigos, sem nenhum `if` por tipo. O comportamento vem do override e dos atributos da instância:

```
# Quem rola o loot NAO sabe o tipo concreto do inimigo:
item = random.choice(inimigo.tabela_loot()) # cada classe devolve o seu pool

# receber_dano e polimorfico:
class Inimigo(Entidade):
    def receber_dano(self, dano):
        # desconta armadura
        return ... max(0, dano - self.acao_dano) ...
class Jogador(Entidade):
    def receber_dano(self, dano):
        # sofre direto
        ...
```

O método `Inimigo.atacar(alvo)` é o auge: a MESMA chamada faz o Nosferatu roubar vida, a Banshee atordoar e um goblin só bater — guiado pelos atributos (`cura_percentual`, `chance_atordoar`), **sem if por tipo**.

Dica: O cheiro de código que o polimorfismo elimina é o `if/elif` por tipo (`if tipo == 'nosferatu': ... elif tipo == 'banshee': ...`). Se cada tipo sabe se comportar, você só chama o método. Adicionar um inimigo novo não mexe no laço de combate — só cria uma subclasse. Isso é extensibilidade.

Parte 3 — O que o Python adiciona

O próprio slide final do professor lista: "Python adiciona herança múltipla, dunder methods, @property e duck typing". Veja cada um e onde (ou se) aparece no Randomeon.

Duck typing

"Se anda como pato e faz quack, é um pato." Em vez de exigir herança, basta o objeto ter o método/atributo certo. O Randomeon usa o idioma defensivo `getattr(obj, 'attr', padrão)`:

```
# Inimigo.atacar: funciona com qualquer "alvo" que exponha o atributo certo
miss = self.chance_miss + getattr(alvo, "evasao_passiva", 0.0)

# Item.usar: so chama curar_veneno se o jogador tiver esse metodo
curar = getattr(jogador, "curar_veneno", None)
if callable(curar): curar()
```

Dica: `getattr(obj, nome, padrão)` lê um atributo com um valor de reserva se ele não existir. Isso tolera variações de objeto (até dummies de teste) sem quebrar e sem exigir herança — duck typing na prática.

Dunder methods (métodos mágicos)

Métodos com `__duplo_underscore__` que o Python chama em certas operações. O slide lista vários (`__init__`, `__str__`, `__repr__`, `__eq__`, `__lt__`, `__len__`, `__add__`, `__contains__`, `__iter__`, `__getitem__`). O Randomeon usa `__init__` (em todas as classes) e `__repr__` para debug:

```
def __repr__(self):
    return (f"Jogador(nome={self.nome!r}, nivel={self.nivel}, "
            f"hp={self.hp}/{self.hp_max}, atk={self.atk}, xp={self.xp})")
```

Os dunder mais ricos (`__eq__`, `__lt__`, `__len__`, `__iter__`, `__getitem__`) não foram necessários até aqui. Encaixe natural, se quiser demonstrar: `BandoDeGoblins` já é uma coleção (`__len__`/`__iter__`) e `Item` poderia ordenar por `bônus` (`__lt__`/`__eq__`).

@property

Já visto no Encapsulamento. No projeto é usado como **getter calculado** (`pontuacao`, `veneno_turnos`, `envenenado`, `ja_renasceu`) — leitura derivada, sem setter (a validação mora no `__init__` e nos métodos).

@staticmethod e @classmethod

Métodos que não dependem de uma instância. O Randomeon usa `@staticmethod` como **fábrica** de inimigos:

```
class Inimigo(Entidade):
    @staticmethod
    def gerar(andar=1):
        # fabrica: sorteia comum/elite/especial
        if random.random() < 0.10: return HordaDeGoblins()
        ... # escala por andar, despacha por nome
        return Orc(hp, atk, xp, moedas)
```

@classmethod (recebe cls, útil para fábricas que usam a própria classe) não foi usado — a fábrica gerar() é @staticmethod por não precisar de cls.

Herança múltipla e MRO

Python permite herdar de várias bases; o MRO (Method Resolution Order) define a ordem de busca. O Randomeon usa **herança simples** (mais clara) — o domínio não pediu misturar duas bases. Encaixe

possível: um mixin Venenoso/Atordoante.

Composição vs Herança

Professor: quando uma classe "É" outra, herde; quando "TEM" outra, componha. O Randomeon tem o exemplo perfeito lado a lado:

```
class Goblin(Inimigo):                                # HERANCA: Goblin E UM Inimigo
    def tabela_loot(self): return LOOT_HORDA

class BandoDeGoblins:                                # COMPOSICAO: o Bando TEM 3 Goblins
    TAMANHO = 3
    def __init__(self):
        self.goblins = [Goblin("Goblin", hp, atk, xp, moedas)
                          for _ in range(self.TAMANHO)]    # NAO e um Inimigo
```

Dica: Pergunte "X É UM Y?" ou "X TEM UM Y?". O Bando não é um inimigo — é um agrupador que TEM inimigos. Compor evita herança forçada e mantém cada responsabilidade no seu lugar. A Masmorra também compõe: ela TEM um Jogador e um GeradorSala.

Parte 4 — Conteúdo ALÉM dos slides

O que o Randomeon usou de POO/Python que os slides NÃO ensinaram. Não são erros — são recursos e padrões que o projeto trouxe para resolver problemas reais. Conhecê-los enriquece a prova.

Anotações de tipo (type hints)

Os slides não cobrem typing, mas o projeto anota tudo: parâmetros, retornos e Optional/list/tuple. Documenta a intenção e ajuda ferramentas a achar erros.

```
def atacar(self, alvo) -> dict: ...
def gerar(andar: int = 1) -> "Inimigo": ...
tipo_especial: Optional[str] = None
```

Padrões de projeto (Design Patterns)

- **Template Method** — `Inimigo.tentar_renascer()`: a base devolve False; só `CoracaoDaMasmorra` sobrescreve para ressuscitar 1x. O laço de combate chama o hook sem saber o tipo (a 2ª fase do boss nasce sem um 'if').
- **Strategy (via objetos de efeito)** — `EfeitoStatus` e subclasses (`Veneno/Fraqueza/EsquivaReduzida`): cada efeito encapsula seu comportamento em hooks; a Entidade processa todos uniformemente.
- **Factory** — `Inimigo.gerar()` centraliza a criação procedural.
- **Value Object** — `Dom (dom.py)`: objeto que sabe se aplicar a um Jogador (`Bruto/Resistente/Ágil/Sortudo/Sanguessuga`).

Validação por exceções

Os construtores levantam `ValueError` para estados inválidos (`hp<=0`, `atk<=0`, chance fora de 0..1). A base abstrata usa `raise NotImplementedError`. É uma disciplina de "falhe cedo, falhe claro" que vai além do único exemplo de `ValueError` do slide.

Constantes de módulo como configuração tunável

Além dos atributos de classe, o projeto usa constantes de módulo (`CHANCE_VENENO`, `ESQUIVA_ORC`, `BOSS_HP_BASE`, `ESPECIAL_HP_MULTIPLICADOR`) separando os números de balanceamento da lógica — calibrados por simulação Monte Carlo.

Arquitetura em camadas (separação de responsabilidades)

O código separa **domínio** (`jogo/`: regras puras, sem print/IO), **API** (`api/`: FastAPI/REST) e **apresentação** (CLI e frontend). O `GeradorSala` só produz dados; quem exhibe é outra camada. Isso é "lógica != apresentação", um princípio além do escopo de classe-única dos slides.

Outros recursos Python no projeto

- **dataclass** com `field(default_factory=list)` no `GameState (api)`.
- **List comprehensions**: `[Goblin(...) for _ in range(3)]`.
- **Argumentos nomeados e com padrão**: `bonus_hp: int = 0, super().__init__(nome=..., hp=...)`.
- **min/max para limites seguros**: `min(self.hp_max, self.hp + q)`.
- **__new__ nos testes**: cria um `Inimigo dummy` pulando o `__init__` (técnica avançada).
- **Mocking** (`unittest.mock.patch`) e **simulação Monte Carlo** para decisões de balanceamento guiadas por dados.
- **Curto-circuito proposital** (`esquiva > 0 and random()...`) para preservar sequências de RNG nos testes seeded.

Dica: Testes (pytest), fixtures, parametrize e mocking não estão nos slides, mas são parte essencial de um projeto OO de verdade: garantem que herança e polimorfismo continuem funcionando a cada mudança. O Randongeon tem 656 testes de jogo + 35 de API.

Parte 5 — Crônica do projeto (o que a pasta docs/ aborda)

Cada lote do projeto tem um documento em docs/ registrando o que mudou e quais pilares foram usados. Resumo do que a pasta cobre, ligando ao POO:

Lote / Tema	Foco de POO	Destaque
A — Balanceamento v3.2	Atributos de classe	Curva de boss/nível em constantes
B / B1 — Entidade(ABC)	Abstração + ABC + Herança	Base abstrata de Jogador e Inimigo
B2 — EfeitoStatus	Polimorfismo (hooks)	Veneno/Fraqueza/EsquivaReduzida
C — Loot por tipo	Polimorfismo (override)	tabela_loot() sem if por tipo
D — API + inventário	Encapsulamento (contrato)	Schemas FastAPI x domínio
E — Bando de Goblins	Composição vs Herança	Bando TEM Goblins; Goblin É UM Inimigo
I — Inimigo.atacar()	Encapsulamento + Polimorfismo	Turno do inimigo unificado, sem if-tipo
M — Veneno (DoT)	Encapsulamento/Polimorfismo	Efeito de dano por turno
1 — Crítico	Encapsulamento	rolar_dano() concentra a regra
2 — Identidade/evasão	Herança/Polimorfismo	Orc/Troll viram subclasses
3 — Dom de slot	Value Object/Encapsulamento	aplicar_dom(jogador, id)
4 — Coração 2 fases	Template Method	tentar_renascer() (hook polimórfico)
5 — Badge de status	Polimorfismo (serialização)	Efeitos expostos por tipo+turnos
Feedback level-up	Encapsulamento	progresso_nivel() + mensagem única
Recalibração geral	Atributos/constantes	Curva de boss tunável (Monte Carlo)
6 — Tutorial / 7 — Auditoria	Documentação	Estado final + aderência aos slides

Outros docs cobrem frontend e correções (placar, robustez de sessão, textos, save em .txt, victory/game-over) — menos densos em POO, mas parte do mesmo projeto. Todo lote foi testado e, quando mexeu em balanceamento, calibrado por simulação.

Parte 6 — Resumo dos pilares + dicas finais

Pilar	Em uma frase	No Randomeon
Abstração	Esconder o COMO, expor o O QUÊ (ABC + @abstractmethod).	Entidade(ABC), receber_dano abstrato
Encapsulamento	Proteger o estado; público/_protegido/___privado + @property.	_atualizar_nivel, @property pontuacao
Herança	Reusar via subclasses; super() chama o pai; filho É UM pai.	Inimigo e 8 subclasses
Polimorfismo	Mesmo método, comportamento por tipo; override sem if-tipo.	atacar(), tabela_loot(), receber_dano()

Dicas finais de estudo

- **Saiba citar com a palavra certa:** ao mostrar um trecho, diga o pilar ("isto é polimorfismo por sobrescrita") — o professor valoriza o vocabulário.
- **Abstração = ABC:** sempre que ver `class X(ABC)` e `@abstractmethod`, é abstração + um contrato.
- **Encapsulamento tem 3 níveis + @property:** lembre público, _protegido, ___privado, e que `@property` faz getter/setter com sintaxe de atributo.
- **Herança x Composição:** teste com "É UM" vs "TEM UM". Goblin É UM Inimigo (herança); Bando TEM Goblins (composição).
- **Polimorfismo bom não tem if-tipo:** o objeto sabe se comportar; você só chama o método.
- **Dunder:** `__init__` (criar), `__repr__`/`__str__` (mostrar), `__eq__`/`__lt__` (comparar/ordenar), `__len__`/`__iter__`/`__getitem__` (coleções).
- **@staticmethod x @classmethod:** `static` não recebe nada de especial (função utilitária/fábrica simples); `class` recebe `cls` (fábrica que usa a própria classe).
- **Para a prova:** tenha um exemplo de cada pilar na ponta da língua — o Randomeon te dá um real para cada um.

Documento de estudo gerado a partir dos slides do professor (POO Slide/aula/Código) e do código-fonte do projeto Randomeon. Use junto com a auditoria em docs/auditoria-poo-prova.md.